Task 1 - Understanding MaxText data formats

The findings for this task are structured in a way where the overview is first explained. Then, the types of format and their working mechanism is analysed and explained. After this, the tradeoffs and important inferences are discussed following which the short summary is also presented.

Overview:

There are seven types of data formats that are directly and indirectly supported by MaxText. Direct support refers to the fact that the particular data format can be read and processed using the logic that already exists in MaxText. Indirect support means MaxText delegates that responsibility to another library, such as Hugging Face Datasets. MaxText tells the Hugging Face library what dataset to load, receives the resulting dataset object, and continues with tokenization, batching, and training. The parsing of these formats is handled by the Hugging Face library, not by MaxText itself. For training any ML model, the format should be chosen according to the requirements and purpose of the model. For large scale distributed training of LLMs, ArrayRecords is the best type that can be chosen due to its working mechanism: random access memory.

Types of formats that exist and how they differ:

The types of formats that exist are ArrayRecords, Parquet, TFRecords, Apache arrow, JSON, CSV, and text. The main difference found in these types: they are either text files or binary files, and have direct or indirect support from MaxText.

ArrayRecords - binary - direct support
Parquet - binary - direct support
TFRecords - binary - direct support
Apache arrow - binary - indirect support
JSON - text - indirect support
CSV - text - indirect support
Text - text - indirect support

How each type works: mechanism

ArrayRecord: Random access to individual records.
Parquet: Stores data by **columns**, enabling efficient reads of selected fields.
TFRecord: Stores records sequentially, one after another.

Apache Arrow: Keeps data in a **columnar in-memory** layout for fast processing.
JSON: Represents data as nested **objects and arrays** (hierarchical structure).
CSV: Represents data as **rows and columns** separated by delimiters (usually commas).
TXT: Stores raw text as a simple sequential stream of characters or lines.

The kinds of trade-offs possible:

1. Data Pipeline and Format Trade-offs

- **Grain (ArrayRecords) vs. Hugging Face (Text/JSON):**

  The Trade-off: Setup complexity vs. At-scale performance.

  - Using Grain with ArrayRecords requires upfront effort to convert data into a binary format. In return, we get high-speed random access, fault tolerance, and deterministic global shuffling. Using Hugging Face with text formats is extremely easy to set up for experimentation but suffers from slow sequential parsing and weak determinism during multi-host distributed training.

2. Parallelism and Sharding Trade-offs

- **Fully Sharded Data Parallelism (FSDP) vs. Tensor Parallelism (TP):**
  - **The Trade-off:** Memory availability vs. Network communication overhead.
  - FSDP shards the model's weights and states to save memory, requiring an All-Gather communication step. It works well if the batch size is large enough to hide the communication latency. TP shares individual matrix calculations; while it reduces the compute load per device, it introduces massive communication overhead that can easily bottleneck the system if the arithmetic intensity is not perfectly tuned.
- **Context Parallelism (CP):**
  - **The Trade-off:** Sequence length vs. Sharding complexity.
  - CP allows us to train on extremely long sequences (e.g., 128k tokens) by sharding the sequence dimension to prevent memory exhaustion, but it introduces significantly more communication overhead than FSDP.

3. Activation Rematerialization (Memory vs. Compute)

- **Aggressive vs. Minimal Rematerialization:**
  - **The Trade-off:** High Bandwidth Memory (HBM) usage vs. Recomputation time.
  - If we apply an aggressive policy (dropping activations after the forward pass), we can free up massive amounts of VRAM to fit larger batch sizes or sequence lengths. The trade-off is that the hardware must recalculate those math operations

during the backward pass, slowing down the step time. Minimal rematerialization runs faster but risks Out-Of-Memory (OOM) crashes.

4. Checkpointing Trade-offs (via Orbax)

- **Background Saving vs. Blocking Save:**
    - **The Trade-off:** Training throughput vs. System resource safety.
    - MaxText uses Orbax for checkpointing. Background saving allows the training loop to continue while weights are written to disk, massively improving speed. But it temporarily consumes additional system memory and I/O bandwidth, whereas blocking ensures complete data safety at the cost of halting the training loop entirely.

5. Mixture-of-Experts (MoE) Routing

- **Token Dropping vs. Dropless:**
    - **The Trade-off:** Training speed vs. Model quality.
    - When training MoE models, the default approach drops tokens that overflow a specific expert's capacity to keep matrix shapes fixed and fast. Implementing dropless routing keeps every token (improving quality) but creates variable tensor shapes that can severely spike memory usage and slow down execution.

6. Hardware Environment

- **Single-Host vs. Multi-Host Clusters:**
    - **The Trade-off:** Development speed vs. Production scale.
    - Single-host setups (1 TPU VM or 1 GPU node) are easy to debug and lack network latency. Multi-host setups are mandatory for models larger than a few billion parameters but require complex orchestration (like Google's JobSet or XPK), topology tuning, and network debugging.

Use cases, reasons and methods:

Type	Best use case	Reason for choosing	Pipeline
ArrayRecord	Production-scale LLM training across CPUs, GPUs, and	Efficient data distribution across multiple training	Grain

	TPUs	hosts.	
Parquet	Large datasets that already exist in Parquet	Its columnar storage reduces storage size and speeds up reading of required columns	Grain, HuggingFace
TFRecord	Existing TensorFlow datasets	It is TensorFlow's native record format and integrates seamlessly with TensorFlow data pipelines	Grain, TensorFlow datasets
Apache arrow	Local preprocessing and Hugging Face workflows	Designed for efficient memory sharing and high speed columnar processing	HuggingFace
JSON	Small datasets and experimentation	Human-readable and easy to create, edit, and debug	HuggingFace
CSV	Tabular datasets	Provides a simple and universally supported way to store structured table data	HuggingFace
TXT	Raw corpora and custom preprocessing pipelines	Provides the simplest way to store unprocessed textual data before tokenization	HuggingFace

Pros and cons:

Type	Pros	Cons
ArrayRecord	1. Very high read performance 2. Random access to records 3. Deterministic and	1. Requires dataset conversion 2. More setup effort 3. Limited adoption outside Google's

	reproducible 4. Fault-tolerant during interrupted training 5. Optimized for distributed training	ecosystem
Parquet	1. Compressed and storage-efficient 2. Fast column-wise reads 3. Industry-standard format 4. Works well with big data tools	1. Not specifically designed for LLM training 2. Random record access is less efficient than ArrayRecord 3. Requires understanding of schema
TFRecord	1. Efficient binary storage 2. Mature and stable 3. Good TensorFlow integration 4. Handles large datasets well	1. Sequential access only 2. Less flexible outside TensorFlow 3. Less suitable for 4. Very large distributed training than ArrayRecord
Apache Arrow	1. Extremely fast in-memory operations 2. Efficient columnar layout 3. Used internally by Hugging Face Datasets 4. Supports interoperability across libraries	1. Primarily an in-memory format 2. Not intended as a production training dataset format 3. Limited direct use by MaxText
JSON	1. Human-readable and easy to modify 2. Flexible hierarchical structure and universally supported	1. Large file sizes and slow parsing 2. Inefficient for large-scale training
CSV	1. Easy to create and human-readable 2. Supported by almost every data tool and has a lightweight format	1. Cannot naturally represent nested data 2. Slower than binary formats and poor scalability for very large datasets

TXT	1. Easy to inspect and edit, no special tools required and extremely simple	1. No inherent structure or metadata and requires preprocessing before training 2. Inefficient for very large datasets and limited support for complex annotations

Important inferences:

Dataset pipelines:

A data pipeline is a complete sequence of steps that transforms raw data into batches fed to the model. It is similar to an assembly line. MaxText supports three primary dataset pipelines: Grain, Hugging Face, and TensorFlow Datasets (TFDS). Grain is the recommended option because it supports ArrayRecord, which provides random access, deterministic loading, and good scalability for distributed training. The Hugging Face pipeline is the easiest to use because it can stream datasets directly from the Hugging Face Hub or local/cloud storage in several formats, but it is less suitable for large multi-host training due to weaker determinism after interruptions. TFDS works with TFRecord datasets and offers good performance, but it only supports TFRecord and relies on sequential access. So, the appropriate dataset_type value depends on where the dataset is stored and the scale of training, with Grain generally being the preferred choice for production workloads.

ArrayRecord is very important for MaxText's scalability, which means Grain is the strategic direction. ArrayRecord's random-access design enables global shuffling and balanced data distribution, avoiding many of the bottlenecks associated with sequential file formats. We use ArrayRecord format because we can skip right to the data we want, without wasting time on the previous records. Since Grain requires a set-up, we can get familiar with HuggingFace initially to focus more on the training workflow. We can then focus on data pipeline optimization by switching to Grain.

Final summary:

MaxText supports seven data formats divided into direct support for binary files (ArrayRecords, Parquet, TFRecords) read natively by the system, and indirect support for text-based or external

formats (JSON, CSV, TXT, Apache Arrow) handled via Hugging Face. While simple text formats are highly useful for quick experimentation and debugging, binary formats are required for scaling up workloads. Specifically, ArrayRecords managed through the Grain data pipeline stand out as the ideal choice for production-scale LLM training because their unique random-access mechanism allows the system to skip directly to any piece of data, preventing severe performance bottlenecks and ensuring fast, balanced data distribution across multiple GPUs or TPUs.